1.1: Scenario/Issue

My client is the parent of a student at my school who helps his son with homework from different classes. Although my client has enough knowledge on most basic subjects, he has difficulty being able to keep up with his son's language classes due to the amount of memorisation needed and not always knowing his son's curriculum, (See interview #1). While interviewing the client, I found out that he has difficulty using language applications such as Duolingo or Babble because they lead someone down a specific path instead of letting them learn what they need, and because they have a learning curve to them to be able to fully utilize their potential. Additionally, he informed me that manual methods of trying to learn are both costly in time and effort for giving very little reward. Hence, I chose to discuss possible solutions for the parent's problem to solve some of the pain of language acquisition.

1.2: Computational Solution

After consulting with the client, it was obvious that a computational solution was most viable because of the nature of the problem. Analog methods of studying such as using paper or flashcards take a large amount of time and do not allow for diversifying methods of studying vocabulary without requiring an obscene amount of effort. Other than the time consumption, the creation of such study methods is incredibly repetitive and difficult to continue for a long time. The automation that is allowed by using a computational solution would cut down the time by a lot and it would allow for much more diverse studying methods. Other than fixing the problems of analog solutions, a custom computational solution would allow the user to make their own decisions of what to learn instead of going along with what the application prescribes for them.

For this application, I will be using C++ with the Qt desktop application framework. I am using C++ because of the fact that it is a multi-paradigm programming language, allowing for procedural, functional, and object oriented programming. This flexibility, as well as the memory management of C++ allows for efficient code. The Qt framework is ideal for a desktop application, being universal across different platforms. For the backend database, I will be using SQLite, a local database which does not take up a lot of space. This is perfect because it allows the user to avoid having a connection without the tradeoff of losing storage. My choice of IDE will be Visual Studio Code as well as Qt Creator. Visual Studio Code allows for many different file types to be edited, while Qt Creator allows for the creation of GUI elements. For the AI integration I will be using OpenAI's framework which allows the use of their models through API keys. Finally, for the spaced repetition system I will be using the algorithm Supermemo, a widely accepted algorithm that is not difficult to implement.

1.3: Success Criteria

1. The application allows the user to create and manage custom vocabulary lists with capabilities of holding over 1000 words in a list
2. The system supports at least 3 world languages such as Spanish, Latin, and German.
3. The application must provide at least 3 testing modes: Showing a word and presenting multiple choice options for its meaning/translation, showing a meaning/translation and requiring the user to provide the word, flashcard study methods.
4. The application supports bidirectional testing, from one language to another and the other way around
5. The system must have a spaced repetition system with an algorithm that gives the user the necessary words to study based on the users confidence score and last date studied
6. Definitions/meanings of words should include at least 3 ways of being expressed: examples, word relations, and parts of speech.
7. The user must be able to view a report/summary of the words/list they have been studying or currently learning
8. The AI integration allows the user to generate new vocabulary lists through prompts of at least 20 words
9. The user can create a list within ten clicks of opening the application
10. The vocabulary lists that the user creates can be accessed after the application is closed.